1. Definition

1.1. Project Overview

Background --> Tomatoes play a very important role in the agricultural sector. They are very widespread due to their demand. Unfortunately, tomatoes are susceptible to diseases. (Singh, Singh and Kumar, 2017) state that tomatoes can be plagued by upwards of 200 diseases. These diseases can vary in severity but almost always they lead to significant financial and production losses.

Problem Domain --> Manual ways to detect which plants have been affected can be tedious and costly . Additionally, it can be very unreliable. However, nowadays there are very robust computer vision models that can automate these processes and can significantly reduce the need for the employees to manually survey the plantations.

Dataset --> Tomato Leaf Disease Detection - YOLOv8 Dataset from Kaggle will be used in this project. Can be found at the link below.

https://www.kaggle.com/datasets/vasantharank/tomato-leaf-disease-detection-yolov8-dataset/data

1.2. Problem Statement

There is a need for automated and robust models that can identify and classify diseases from images of leaves in a reliable and timely manner. Current methods of classification can be inefficient as they are costly and slow. These inefficiencies can allow diseases to spread and affect the whole crop. This consequently can cause shortages and affect the financial ability of the sector.

1.3. Solution Statement

Goal --> Develop, tune and evaluate the latest YOLOv8 pre-trained model for classification of 9 classes of diseases with 1 class of healthy leaves.

Solution --> Fine Tuned YOLOv8 model trained on the Tomato dataset. YOLOv8 is exceptional in real-time object detection. It is highly balanced in terms of accuracy and speed. There are multiple types of models starting from nano all the way to the x-large this provides scalability. Frameworks are very to use and are optimized to work on multiple platforms. The model is expected to take standard image as "jpg" files and provide output in the four detected objects as bounding boxes along with class the image belongs to.

Future Deployment Options --> After training the model it is possible to deploy the model on mobile devices for people to use in the fields to accurately identify diseased leaves.

1.4. Evaluation Metrics

Metrics Chosen: --> Precision, Recall, F1-Score, Mean Average Precision(mAP) and Time.

Justifications

Precision --> By monitoring precision it is possible to minimize false alarms which can accumulate additional labor costs.

Recall --> By monitoring recall it is possible to minimize missed detections which can lower the productivity of the model.

F1-Score --> By monitoring f1-score it is possible to balance precision and recall getting meaningful evaluation.

mAP --> Is a standard object detection metric including localization and classification accuracy.

Time --> Object detection models usually use FPS as a metric to evaluate time. The lowest threshold for this metric will be 15 FPS.

2. Analysis

2.1. Data Exploration

Dataset Description --> The dataset is a collection of images of tomato leaves consisting of 10 classes the data was collected from kaggle. Data originally was resized, however, not augmented. Dataset originally was created with YOLOv8 model in mind and has a structure that fits the model perfectly.

Dataset Link --> https://www.kaggle.com/datasets/vasantharank/tomato-leaf-disease-detection-yolov8-dataset/data)

Dataset Size --> Total 10853 images.

Data Split --> 72% Train (7842) || 18% Validation (1960) || 10% Test (1051)

Data Classes -->

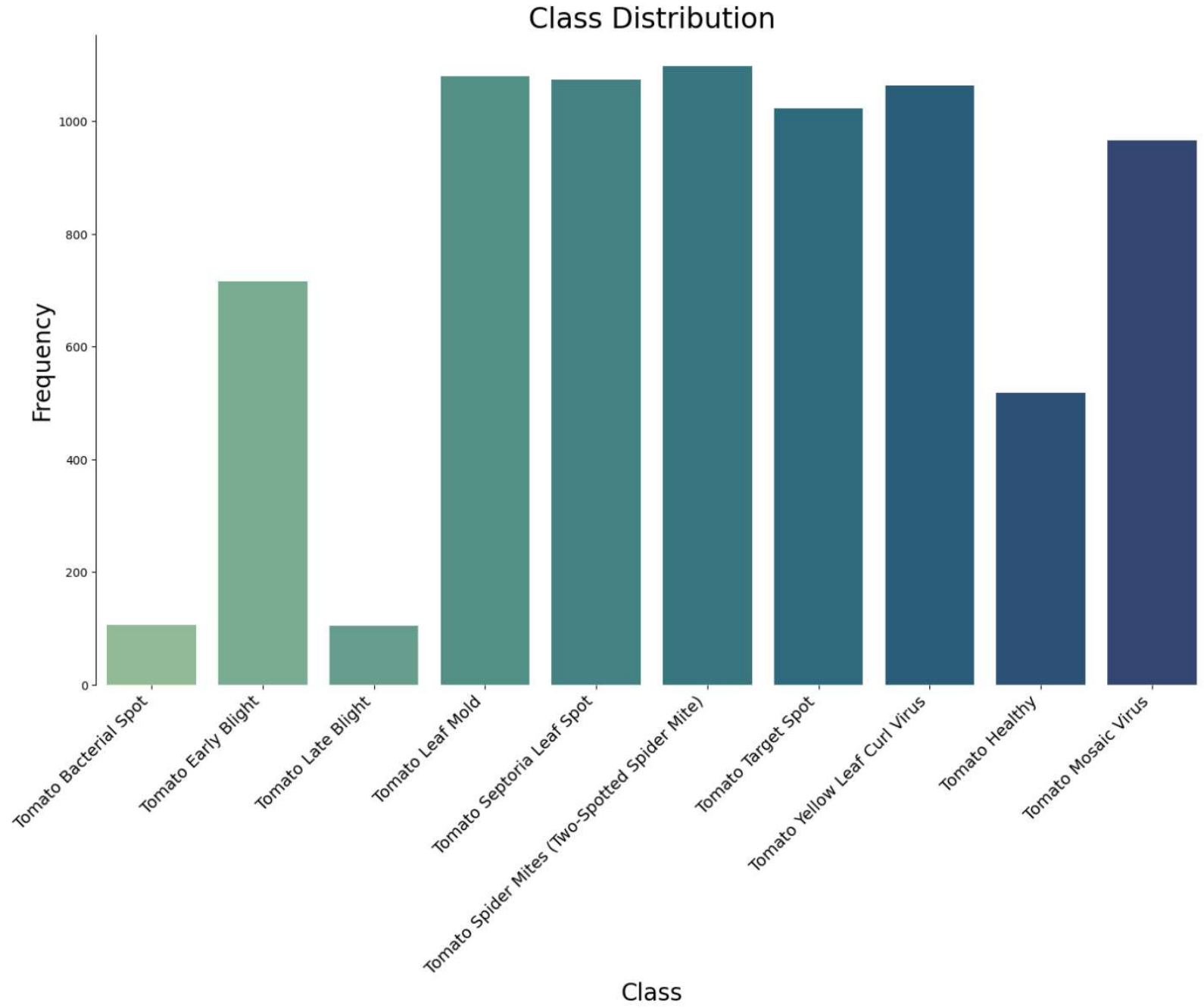
- Tomato Bacterial Spot
- Tomato Early Blight
- Tomato Late Blight
- Tomato Leaf Mold
- Tomato Septoria Leaf Spot
- Tomato Spider Mites (Two-Spotted Spider Mite)
- Tomato Target Spot
- Tomato Yellow Leaf Curl Virus
- Tomato Healthy
- Tomato Mosaic Virus

Data Image Size --> 640x640 pixels in JPG format.

Data Annotation --> YOLOv8 format

2.2. Exploratory Visualization Explanation

The bar chart above shows the class distribution in the dataset. Some classes are under-representing such as bacterial spot and late blight. While the imbalance might affect the training results the model will still provide meaningful results as other classes are approximately balanced.


Class Distribution
Frequency
0
200
400
600
800
1000
Tomato Bacterial Spot
Tomato Early Blight
Tomato Late Blight
Tomato Leaf Mold
Tomato Septoria Leaf Spot
Tomato Spider Mites (Two-Spotted Spider Mite)
Tomato Target Spot
Tomato Yellow Leaf Curl Virus
Tomato Healthy
Tomato Mosaic Virus
Class

2.3. Algorithms and Techniques

Main algorithm that will handle the training is YOLOv8 pre trained object detection model. This model is very recent and is very efficient in real-time object detection as it balances both accuracy and speed so there are nearly no trade-offs. Due to the model architecture the image is processed at once and provides bounding boxes for detected objects. Additional data augmentation techniques are present in the training library and will be used to reduce overfitting and enable the model to generalize better.

2.4. Benchmarking

To compare our trained model to the base model YOLOv5 will be used as a benchmark. YOLOv5 is a predecessor to the YOLOv8 and was used widely as well. Training YOLOv5 on the tomato dataset while providing the same hyperparameters will provide a good baseline model for comparison and enable us to get better understanding on improvements and shortcomings.

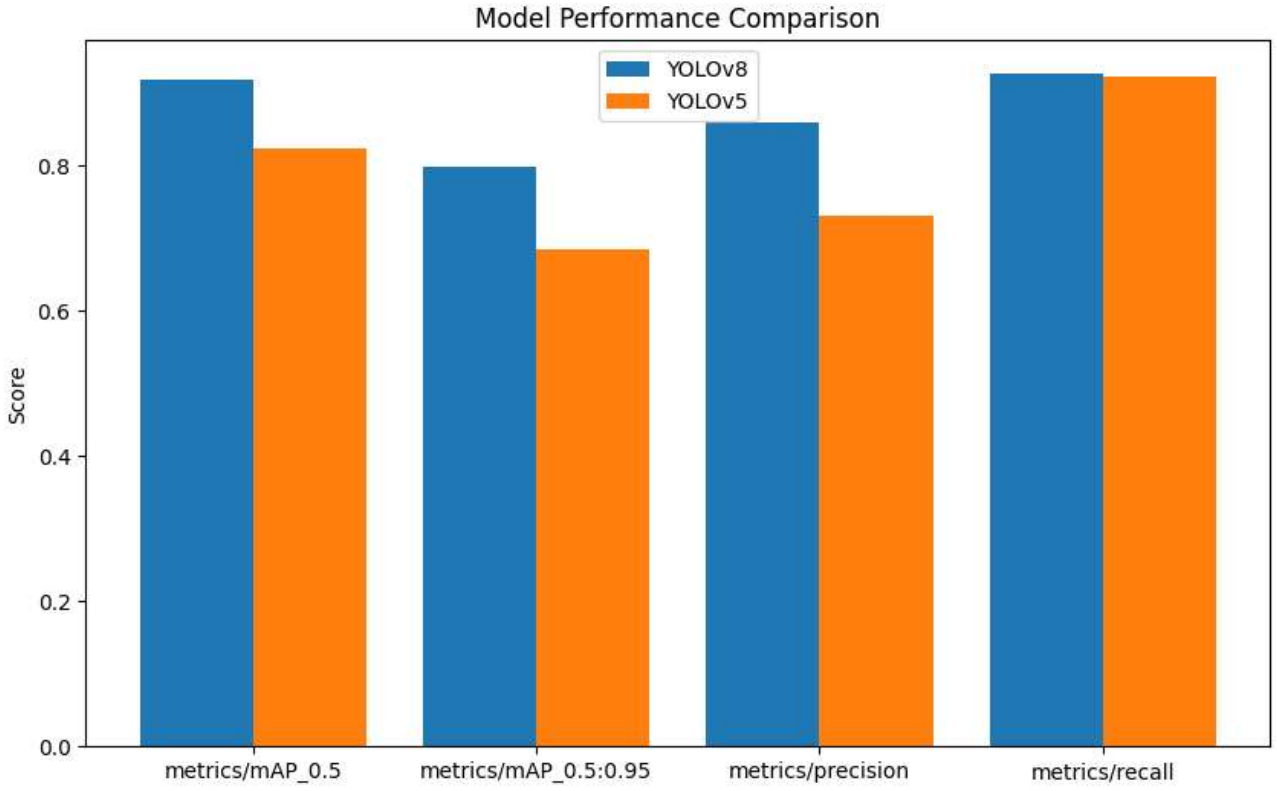
3. Methodology

3.1. Preprocessing

Dataset originally had images at a consistent size of (640x640 pixels) the format and structure were already applicable for YOLO models. So, no extra preprocessing steps are needed. However, ultralytics library automatically applies certain augmentations pre-training such as horizontal flipping, mosaic, combining images, color adjustments. These can help the model to learn better.

3.2. Implementation

The code is implemented with python 3.10 in Jupyter notebook and a clean environment through conda. Ultralytics library was used for training and evaluating both YOLOv5 and YOLOv5 models. The training data was passed through the data.yaml file created earlier. Data.yaml specifies paths for the splits. Hyperparameters were set at --. The training was done with a GPU locally (NVIDIA GeForce RTX 3060). Progress of the model was monitored via the console and logs.



3.3. Refinement

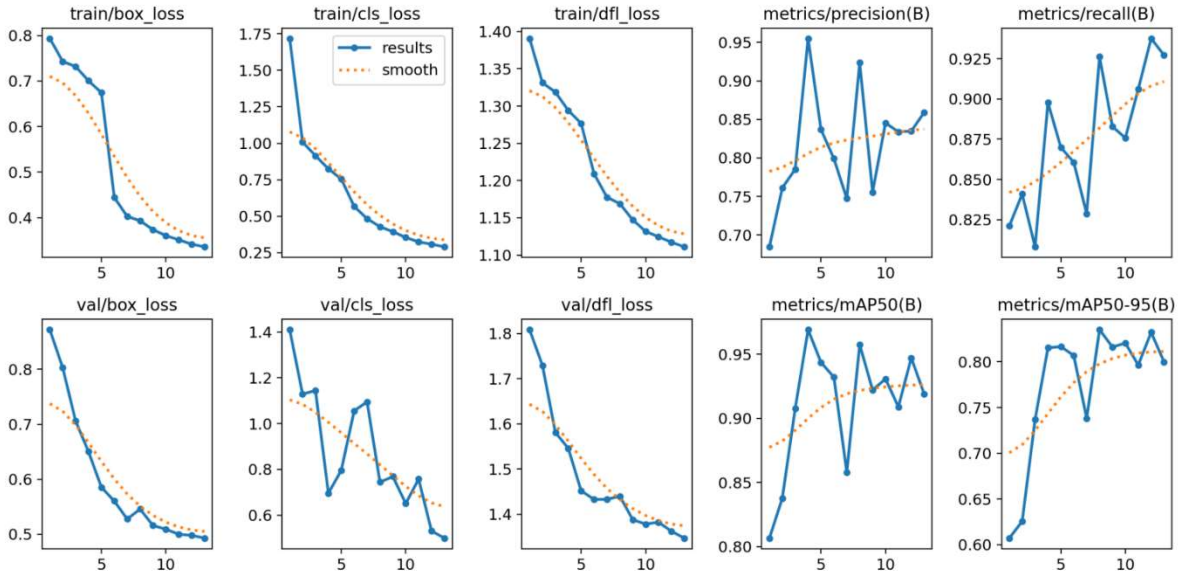
For experimentation purposes new hyperparameters were provided. This time the model was changed to use smaller YOLO model "nano". Epochs decreased to 10 batch size was increased. This would provide an insight into whether the model can be made smaller but still provide good results.

4. Results

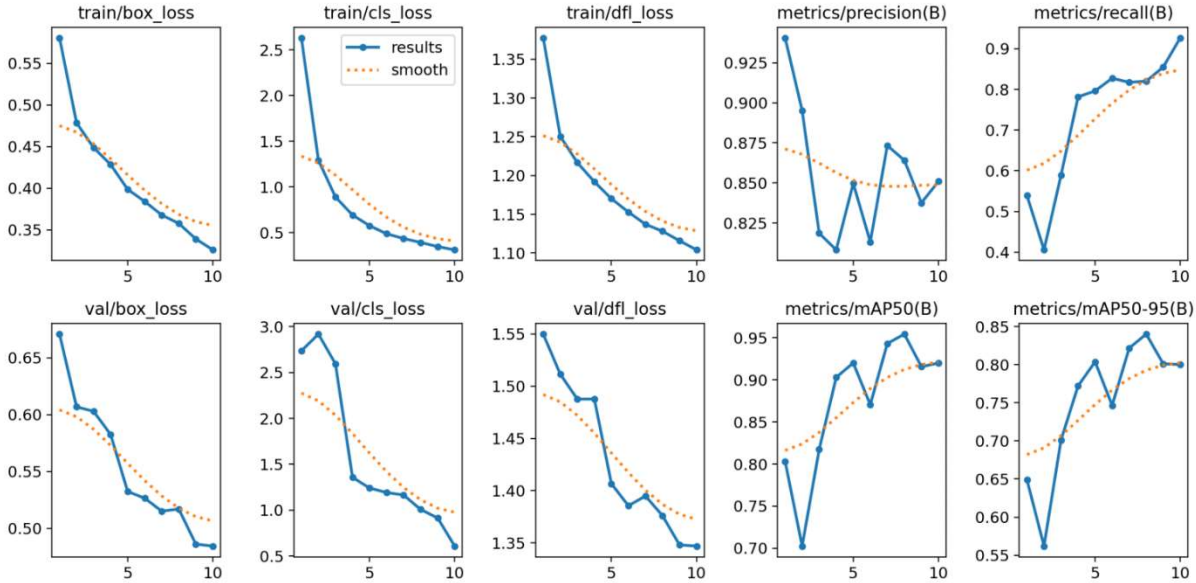
4.1. Model Evaluation

This section will evaluate the results from 3 models original, tuned and benchmark. Compare them and visualize the differences. Refined vs Original --> vs Benchmark

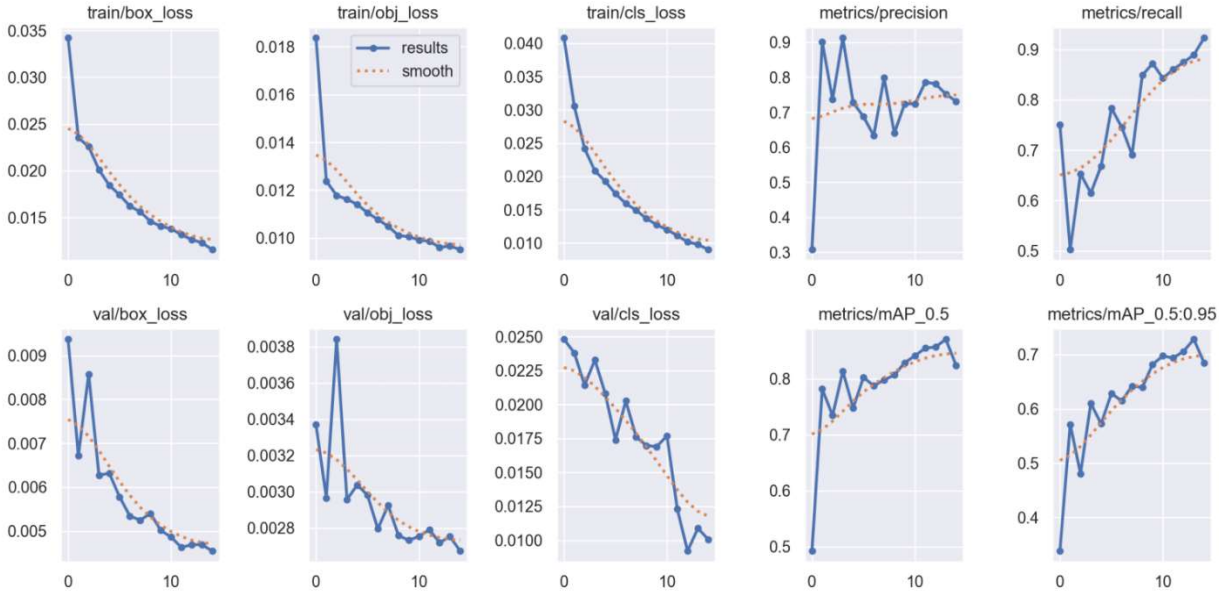
Original YOLOv8 Curves --> (yolov8s_tomato_light)



Tuned YOLOv8 Curves --> (yolov8s_tomato_tuned)



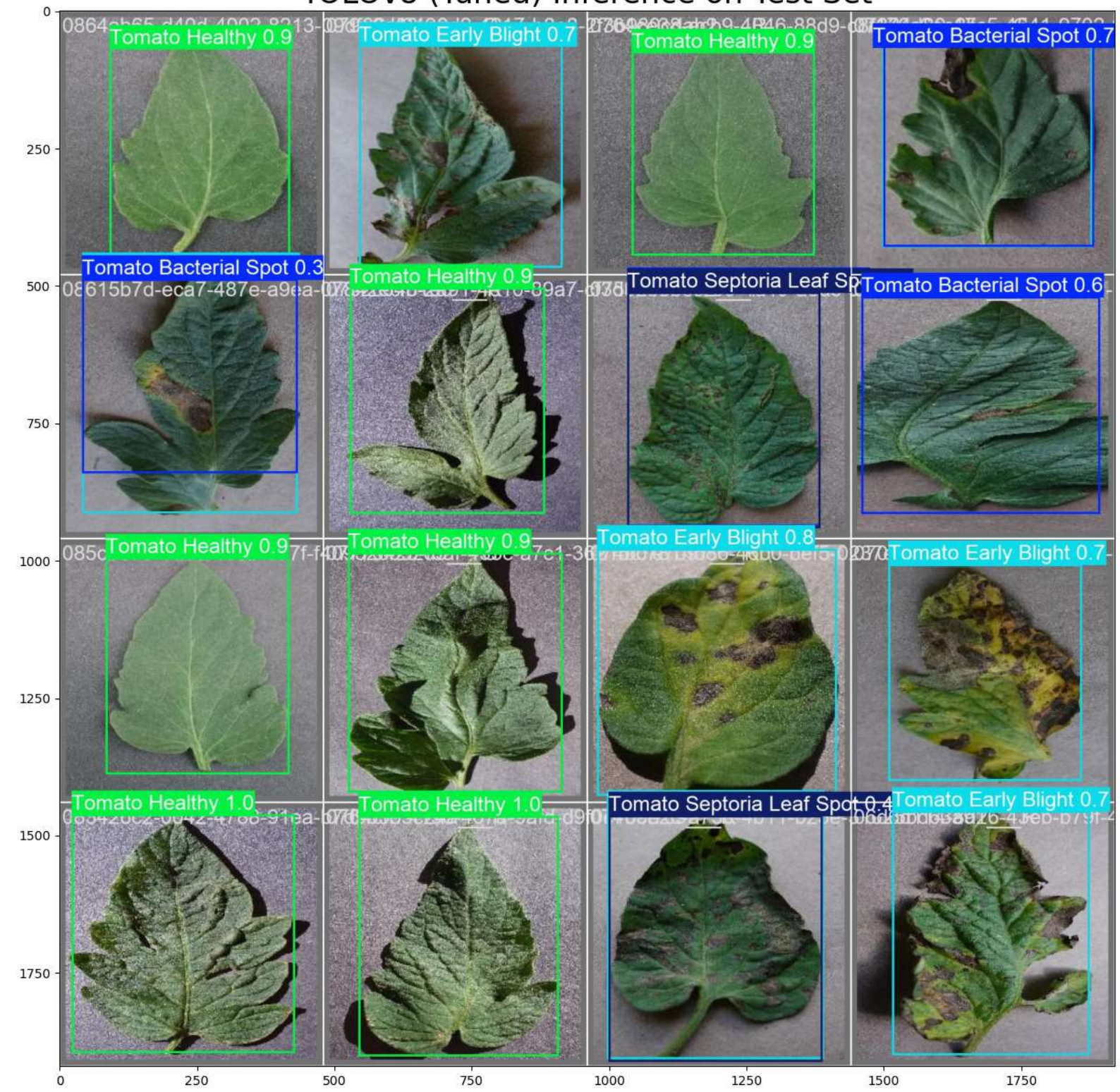
Benchmark YOLOv5 Curves --> (yolov5s_tomato_benchmark_minimal)



4.2. Validation Discussion

Training curves provide clear insight into how the models trained over the epochs on the training and validation sets. The curves for the tuned model show that the model performs reasonably well. Losses can be seen to be decreasing in the first three boxes showing that the model has fitted the data over the course of 10 epochs. Validation also decreases for the duration of training. This shows that the model generalizes well with the unseen data. Accuracy metrics such mAP50 and mAP5095 have the same positive trend while stabilizing towards the end of training they show high results; stabilization is expected due to convergence of the model.

According to the results original YOLOv8 model achieved a mAP score of 0.95 while the tuned model achieved nearly identical results however the benchmark model underperformed slightly. However, for mAP 50 95 part Tuned model performed the best at 0.93 original being close behind and benchmark test being the lowest. This means that the original model being the best performs slightly better than the tuned model. Tuned model is more applicable as it is trained on nano version which is smaller and lightweight meaning it can more easily deployed to mobile/edge devices. Therefore, the best model for this project is tuned model with YOLOv8n the nano version.


YOLOv8 (Tuned) Inference on Test Set
Tomato Healthy 0.9
Tomato Early Blight 0.7
Tomato Healthy 0.9
Tomato Bacterial Spot 0.7
Tomato Bacterial Spot 0.3
Tomato Healthy 0.9
Tomato Septoria Leaf Sp
Tomato Bacterial Spot 0.6
Tomato Healthy 0.9
Tomato Healthy 0.9
Tomato Early Blight 0.8
Tomato Early Blight 0.7
Tomato Healthy 1.0
Tomato Healthy 1.0
Tomato Septoria Leaf Spot 0.4
Tomato Early Blight 0.7
0
250
500
750
1000
1250
1500
1750

5. Conclusion

5.1. Summary

This project successfully handled the creation of the model that can classify tomato leaf health across 10 different classes with Deep learning computer vision model YOLOv8. The pre-trained model was fine-tuned on the tomato dataset. The training consisted of 3 different models, one original one tuned and one benchmark. The tuned model was the smallest among all the other models due to the size choice of "nano".

Model	Test mAP@0.5	Test mAP@0.5:0.95
YOLOv8 (Original)	0.956523	0.826437
YOLOv8n (Refined)	0.954023	0.839756
YOLOv5s (Benchmark)	0.924000	0.688000

This would make it applicable for smaller storage devices. The Benchmark model was YOLOv5, the older version of the pre-trained model. It was trained under the same hyperparameters as the original YOLOv8. After testing the models on the unseen data. Original and tuned models had the same results while the benchmark model underperformed. The tradeoff of size of the model for little difference in 0.1 mAP is inefficient therefore, tuned YOLOv8 was chosen to be the best model.

References

Vasantharan K. (2023). Tomato Leaf Disease Detection - YOLOv8 Dataset. Kaggle. https://www.kaggle.com/datasets/vasantharank/tomato-leaf-disease-detection-yolov8-dataset/data ####

Jocher, G. et al. (2023). Ultralytics YOLOv8. GitHub. https://github.com/ultralytics/ultralytics

Jocher, G. et al. (2020-2023). YOLOv5 by Ultralytics. GitHub. https://github.com/ultralytics/yolov5

Singh, V.K., Singh, A.K. & Kumar, A. (2017). Disease management of tomato through PGPB: current trends and future perspective. *3 Biotech* 7, 255.

https://doi.org/10.1007/s13205-017-0896-1